Appendix A Syntax Reference

The machine-readable dialect of CSP (CSP_M) is one result of a research effort with the primary aim of encouraging the creation of tools for CSP. **FDR** was the first tool to utilise the dialect, and to some extent **FDR** and CSP_M continue to evolve in parallel, but the basic research results are publically available (see [Acknowledgements], page 2). The language described here is that implemented by the 2.1 release of **FDR** and has many features not present in **FDR1**.

CSP_M combines the CSP process-algebra with an expression language which, while inspired by languages like Miranda/Orwell and Haskell/Gofer, has been adapted to support the idioms of CSP. The fundamental features of those languages are, however, retained: the lack of any notion of assignment, the ability to treat functions as first-class objects, and a lazy reduction strategy.

Scripts

Programming languages are used to describe algorithms in a form which can be executed. CSP_M includes a functional-programming language, but its primary purpose is different: it is there to support the description of parallel systems in a form which can be automatically manipulated. CSP_M scripts should, therefore, be regarded as defining a number of processes rather than a program in the usual sense.

A.1 Expressions

At a basic level, a CSP_M script defines processes, along with supporting functions and expressions. CSP draws freely on mathematics for these supporting terms, so the CSP_M expression-language is rich and includes direct support for sequences, sets, booleans, tuples, user-defined types, local definitions, pattern-matching and lambda terms.

We will use the following variables to stand for expressions of various types.

m, n	numbers
s, t	sequences
a, A	sets (the latter a set of sets)
b	boolean
p, q	processes
e	events
c	channel
x	general expression

When writing out equivalences, `z` and `z'` are assumed to be fresh variables which do not introduce conflicts with the surrounding expressions.

A.1.1 Identifiers

Identifiers in CSP_M begin with an alphabetic character and are followed by any number of alphanumeric characters or underscores optionally followed by any number of prime characters (`'`). There is no limit on the length of identifiers and case is significant. Identifiers with a trailing underscore (such as '`fnargle_`') are reserved for machine-generated code such as that produced by Casper [Lowe97].

CSP_M enforces no restrictions on the use of upper/lower case letters in identifiers (unlike some functional languages where only datatype constructors can have initial capital letters). It is, however, common for users to adopt some convention on the use of identifiers. For example

- Processes all in capitals (BUTTON, ELEVATOR_TWO)
- Types and type constructors with initial capitals (User, Dial, DropLine)
- Functions and channels all in lower-case (sum, reverse, in, out, open_door)

Note that while it is reasonable to use single character identifiers ('`P`', '`c`', '`T`') for small illustrative examples, real scripts should use longer and more descriptive names.

A.1.2 Numbers

Syntax

`12`	integer literal
m`+`n, m`-`n	sum and difference
`-`m	unary minus
m`*`n	product
m`/`n, m`%`n	quotient and remainder

Remarks

Integer arithmetic is defined to support values between -2147483647 and 2147483647 inclusive, that is those numbers representable by an underlying 32-bit representation (either signed or twos-complement). The effect of overflow is not defined: it may produce an error, or it may silently wrap in unpredictable ways and so should not be relied upon.

The division and remainder operations are defined so that, for $n \neq 0$,

$$\begin{aligned} m &= n*(m/n)+m\%n \\ |m\%n| &< |n| \\ m\%n &\geq 0 \quad \text{(provided } n > 0\text{)} \end{aligned}$$

This states that for positive divisors, division rounds down and the remainder operation yields a positive result.

Floating point numbers (introduced experimentally for Pravda [Lowe93]) are not currently supported by **FDR**. Although the syntax for them is still enabled, it is not documented here. Note: the dot syntax used in communications (see Section A.3 [Types], page 52) can make an expression *look* just like a floating-point number would in other languages (e.g., '`3.1`' is actually the pairing of two integers — the dot is not a decimal point).

The new unary minus operator can cause some confusion with comments (see Section B.1 [Changes from FDR1 to FDR2], page 64).

A.1.3 Sequences

Syntax

`<>`, `<1,2,3>`	sequence literals
`<`m`..`n`>`	closed range (from integer m to n inclusive)
`<`m`..>`	open range (from integer m upwards)
s`^`t	sequence catenation
`#`s, `length(`s`)`	length of a sequence
`null(`s`)`	test if a sequence is empty
`head(`s`)`	give first element of a non-empty sequence
`tail(`s`)`	give all but the first element of a non-empty sequence
`concat(`s`)`	join together a sequence of sequences
`elem(`x`,`s`)`	test if an element occurs in a sequence
`<`x_1`, ···`x_n `|` x`<-`s`, `b`>`	comprehension

Equivalences

```
        null(s) ≡ s==<>
        <m..n> ≡ if m<=n then <m>^<m+1..n> else <>
      elem(x,s) ≡ not null(< z | z<-s, z==x >)
          < x | > ≡ < x >
    < x | b, ···> ≡ if b then < x | ···> else <>
< x | x'<-s, ···> ≡ concat(< < x | ···> | x'<-s >)
```

Remarks

All the elements of a sequence must have the same type. `concat` and `elem` behave as if defined by

```
concat(s)      = if null(s) then <> else head(s)^concat(tail(s))
elem(_, <>)    = false
elem(e, <x>^s) = e==x or elem(e,s)
```

Similarly, we can define `palindrome` to test if a sequence is its own reverse (that is '`s == reverse(s)`') by

```
palindrome(<x>^s^<y>) = x==y and palindrome(s)
palindrome(_)         = true
```

A.1.4 Sets

Syntax

`{}`	set literals
`{`m`..`n`}`	closed range (from integer m to n inclusive)
`{`m`..}`	open range (from integer m upwards)
`union(`a_1`,`a_2`)`	set union
`inter(`a_1`,`a_2`)`	set intersection
`diff(`a_1`,`a_2`)`	set difference
`Union(`A`)`	distributed union
`Inter(`A`)`	distributed intersection (A must be non-empty)
`member(`x`,`a`)`	membership test
`card(`a`)`	cardinality (count elements)
`empty(`a`)`	check for empty set
`set(`s`)`	convert a sequence to a set
`seq(`s`)`	convert a set to a sequence (in arbitrary order)
`Set(`a`)`	all subsets of a (powerset construction)
`Seq(`a`)`	set of sequences over a (infinite unless a is empty)
`{`x_1`, ···`x_n` | `x`<-`a`, `b`}`	comprehension

Equivalences

```
   union(a1,a2) ≡ { z,z' | z<-a1, z'<-a2 }
   inter(a1,a2) ≡ { z | z<-a1, member(z,a2) }
    diff(a1,a2) ≡ { z | z<-a1, not member(z,a2) }
      Union(A) ≡ { z | z'<-A, z<-z' }
   member(x,a) ≡ not empty({ z | z<-a, z==x })
        Seq(a) ≡ union({ <> }, { <z>^z' | z<-a, z'<-Seq(a) })
        { x | } ≡ { x }
    { x | b, ···} ≡ if b then { x | ···} else {}
{ x | x'<-a, ···} ≡ Union( { { x | ···} | x'<-a } )
```

Remarks

In order to remove duplicates, sets need to compare their elements for equality, so only those types where equality is defined may be placed in sets. In particular, sets of processes are not permitted. See the section on pattern-matching for an example of how to convert a set into a sequence by sorting.

Sets with a leading unary minus (most commonly, sets of negative numbers, '`{ -2}`') require a space between the opening bracket and minus sign to prevent it being confused with a block comment (see Section A.7 [Mechanics], page 62).

A.1.5 Booleans

Syntax

`true`, `false`	boolean literals
b_1 `and` b_2	boolean and (shortcut)
b_1 `or` b_2	boolean or (shortcut)
`not` b	boolean not
x_1`==`x_2, x_1`!=`x_2	equality operations
x_1`<`x_2, x_1`>`x_2, x_1`<=`x_2, x_1`>=`x_2	ordering operations
`if` b `then` x_1 `else` x_2	conditional expression

Equivalences

$$
\begin{aligned}
b_1 \texttt{ and } b_2 &\equiv \texttt{if } b_1 \texttt{ then } b_2 \texttt{ else false} \\
b_1 \texttt{ or } b_2 &\equiv \texttt{if } b_1 \texttt{ then true else } b_2 \\
\texttt{not } b &\equiv \texttt{if } b \texttt{ then false else true}
\end{aligned}
$$

Remarks

Equality operations are defined on all types except those containing processes and functions (lambda terms).

Ordering operations are defined on sets, sequences and tuples as follows

$$
\begin{aligned}
x_1 \texttt{ >= } x_2 &\equiv x_2 \texttt{ <= } x_1 \\
x_1 \texttt{ < } x_2 &\equiv x_1 \texttt{ <= } x_2 \texttt{ and } x_1 \texttt{ != } x_2 \\
a_1 \texttt{ <= } a_2 &\equiv a_1 \text{ is a subset of } a_2 \\
s_1 \texttt{ <= } s_2 &\equiv s_1 \text{ is a prefix of } s_2 \\
(x_1\texttt{,}y_1) \texttt{ <= } (x_2\texttt{,}y_2) &\equiv x_1 \texttt{ < } x_2 \texttt{ or } (x_1 \texttt{ == } x_2 \texttt{ and } y_1 \texttt{ <= } y_2)
\end{aligned}
$$

Ordering operations are not defined on booleans or user-defined types.

In the conditional expression,

```
if b then x1 else x2
```

the values `x1` and `x2` must have the same type. This is partially enforced by the parser. For example,

```
if b then {1} else <2>
```

is a parse error.

A.1.6 Tuples

Syntax

`(1,2)`, `(4,<>,{7})` pair and triple

Remarks

Function application also uses parentheses, so functions which take a tuple as their argument need two sets of parentheses. For example the function which adds together the elements of a pair can be written either as

```
plus((x,y)) = x+y
```

or as

```
plus(p) = let (x,y) = p within x + y
```

The same notation is used in type definitions to denote the corresponding product type. For example, if we have

```
nametype T = ({0..2},{1,3})
```

then `T` is

```
{ (0,1), (0,3), (1,1), (1,3), (2,1), (2,3) }
```

A.1.7 Local definitions

Definitions can be made local to an expression by enclosing them in a '`let within`' clause.

```
primes =
  let
    factors(n)  = < m | m <- <2..n-1>, n%m == 0 >
    is_prime(n) = null(factors(n))
  within < n | n <- <2..>, is_prime(n) >
```

Local definitions are mutually recursive, just like top-level definitions. Not all definitions can be scoped in this way: channel and datatype definitions are only permitted at the top-level. Transparent definitions can be localised, and this can be used to import **FDR2**'s compression operations on a selective basis. For example,

```
my_compress(p) =
  let
    transparent normal, diamond
  within normal(diamond(p))
```

A.1.8 Lambda terms

Syntax

\ x_1 , $\cdots x_n$ @ x lambda term (nameless function)

Equivalences

The definition

```
f(x,y,z) = x+y+z
```

is equivalent to the definition

```
f = \ x, y, z @ x+y+z
```

Remarks

There is no direct way of defining an anonymous function with multiple branches. The same effect can be achieved by using a local definition and the above equivalence. Functions can both take functions as arguments and return them as results.

```
map(f)(s) = < f(x) | x <- s >
twice(n)  = n*2
assert map(\ n @ n+1)(<3,7,2>) == <4,8,3>
assert map(map(twice))(< <9,2>, <1> >) == < <18,4>, <2> >
```

A.2 Pattern Matching

Many of the above examples made use of pattern matching to decompose values. The version of CSP_M used by **FDR** 2.1 introduced much better support for pattern-matching; for example, we can write

```
reverse(<>)    = <>
reverse(<x>^s) = reverse(s)^<x>
```

rather than

```
reverse(s) = if null(s) then <> else reverse(tail(s)) ^ <head(s)>
```

The branches of a function definition must be adjacent in the script, otherwise the function name will be reported as multiply defined.

Patterns can occur in many places within CSP_M scripts

- Function definitions (`reverse` above)
- Direct definitions '`(x,y)=(7,2)`'
- Comprehensions '`{ x+y | (x,y) <- {(1,2),(2,3)} }`'
- Replicated operators '`||| (x,y):{(1,2),(2,3)} @ c!x+y->STOP`'
- Communications '`d?(x,y)->c!x+y->STOP`'

The patterns which are handled in these cases are the same, but the behaviour in the first two cases is different. During comprehensions, replicated operators and communications we can simply discard values which fail to match the pattern: we have a number of such values to consider so this is natural. When a function fails to match its argument (or a definition its value) silently ignoring it is not an option so an error is raised. On the other hand, functions can have multiple branches (as in the case of `reverse`) which are tried in top to bottom order while the other constructs only allow a single pattern. For example,

```
f(0,x) = x
f(1,x) = x+1
print f(1,2) -- gives 3
print f(2,1) -- gives an error
print { x+1 | (1,x) <- { (1,2), (2,7) } } -- gives {3}
```

The space of patterns is defined by

1. Integer literals match only the corresponding numeric value.
2. Underscore ('`_`') always matches.
3. An identifier always matches, binding the identifier to the value.
4. A tuple of patterns is a pattern matching tuples of the same size. Attempting to match tuples of a different size is an error rather than a match failure.
5. A simple sequence of patterns is a pattern (`<x,y,z>`) matching sequences of that length.
6. The catenation of two patterns is a pattern matching a sequence which is long enough, provided at least one of the sub-patterns has a fixed length.
7. The empty set is a pattern matching only empty sets.
8. A singleton set of a pattern is a pattern matching sets with one element.
9. A datatype tag (or channel name) is a pattern matching only that tag.
10. The dot of two patterns is a pattern. (`A.x`)
11. The combination of two patterns using `@@` is a pattern which matches a value only when both patterns do.
12. A pattern may not contain any identifier more than once.

For example, `{}`, `({x},{y})` and `<x,y>^_^<u,v>` are valid patterns. However, `{x,y}` and `<x>^s^t` are not valid patterns since the decomposition of the value matched is not uniquely defined. Also `(x,x)` is not a valid pattern by the last rule: the effect that this achieves in some functional languages requires an explicit equality check in CSP_M.

When a pattern matches a value, all of the (non-tag) identifiers in the pattern are bound to the corresponding part of the value.

The fact that tags are treated as patterns rather than identifiers can cause confusion if common identifiers are used as tags. For example, given

```
channel n : {0..9}
f(n) = n+1
```

attempting to evaluate the expression `f(3)` will report that the function `\ n @ n+1` does not accept the value 3. (It accepts *only* the tag `n`.) This conflict between pattern matching and tags commonly manifests itself when parameterising a process with channels:

```
channel in,out,mid:Int

COPY(in,out) = in?x -> out!x -> COPY(in,out)

BUFF  = COPY(in,out)
BUFF2 = COPY(in,mid) [|{|mid|}|] COPY(mid,out)  -- error!
```

In the definition of `COPY`, the parameters are pattern-matched to be the channels `in` and `out` and so are *not* new identifiers that get bound to the actual arguments. So, the error with the definition of `BUFF2` is that `COPY` is being applied outside its domain — in the first application, `mid` does not match the pattern `out`, since `out` is also a channel name so it matches only that channel.

Only names defined as tags are special when used for pattern-matching. For example, given

```
datatype T = A | B
x = A
f(x) = 0
f(_) = 1
g(A) = 0
g(_) = 1
```

then `f` is not the same as `g` since `f(B)` is `0` while `g(B)` is `1`.

The singleton-set pattern allows us to define the function which picks the unique element from a set as

```
pick({x}) = x
```

This function is surprisingly powerful. For example, it allows us to define a `sort` function from sets to sequences.

```
sort(f,a) =
  let
    below(x)  = card( { y | y<-a, f(y,x) } )
    pairs     = { (x, below(x)) | x <- a }
    select(i) = pick({ x | (x,n)<-pairs, i==n })
  within < select(i) | i <-<1..card(a)> >
```

where the first argument represents a '`<=`' relation on the elements of the second. Because `pick` works only when presented with the singleton set, the `sort` function is defined only when the function `f` provides a total-ordering on the set `a`.

A.3 Types

A.3.1 Simple types

Types are associated at a fundamental level with the set of elements that the type contains. Type expressions can occur only as part of the definition of channels or other types, but the name of a type can be used anywhere that a set is required.

For example, the type of integer values is `Int` and the type of boolean values is `Bool`, so

```
{0..3} <= Int
{true, false} == Bool
```

`Proc` is the type of Processes, so processes can be put into datatypes:

```
datatype D = P.Proc | I.Int
```

In type expressions the tuple syntax denotes a product type and the dot operation denotes a composite type so that

`({0,1},{2,3})` denotes `{(0,2),(0,3),(1,2),(1,3)}`
`{0,1}.{2,3}` denotes `{0.2, 0.3, 1.2, 1.3}`

The `Set` and `Seq` functions which return the powerset and sequence-space of their arguments are also useful in type expressions.

A.3.2 Named types

Nametype definitions associate a name with a type expression, meaning that '`.`' and '`( , , )`' operate on it as type constructors rather than value expressions. The type name may not take parameters.

For example,

```
nametype Values = {0..199}
nametype Ranges = Values . Values
```

has the same effect as

```
Values = {0..199}
Ranges = { x.y | x<-Values, y<-Values }
```

Note that outside of the 'top-level' of a nametype (or datatype, or subtype) definition, the expression '`Values . Values`' has the entirely different meaning of two copies of the set `Values` joined by the infix dot. Similarly the expression '`(Values,Values)`' means the cartesian product of `Values` for the construction of a type, but a pair of two sets in all other contexts.

A.3.3 Datatypes

Syntax

`datatype T = A.{0..3} | B.Set({0,1}) | C`	definition of type
`A.0`, `B.{0}`, `B.{0,1}`, `C`	four uses of type

Remarks

Datatypes may not be parameterised (`T` may not have arguments).

The `datatype` corresponds to the variant-record construct of languages like Pascal. At the simplest level it can be used to define a number of atomic constants

```
datatype SimpleColour = Red | Green | Blue
```

but values can also be associated with the tags

```
Gun = {0..15}
datatype ComplexColour = RGB.Gun.Gun.Gun | Grey.Gun | Black | White
```

Values are combined with '`.`' and labelled using the appropriate tag, so that we could write

```
make_colour((r.g.b)@@x) =
  if r!=g or g!=b then RGB.x else
  if r==0 then Black else
  if r==15 then White else Grey.r
```

to encode a colour as briefly as possible.

Note that while it is possible to write

```
datatype SlowComplexColour = RGB.{r.g.b | r<-Gun, g<-Gun, b<-Gun} | ...
```

this is less efficient and the resulting type must still be rectangular, i.e., expressible as a simple product type. Hence it is *not* legal to write

```
datatype BrokenComplexColour = -- NOT RECTANGULAR
  RGB.{r.g.b | r<-Gun, g<-Gun, b<-Gun, r+g+b < 30 } | ...
```

A.3.4 Subtypes

Syntax

`subtype U = A.{0,2} | C`	definition of subtype

Remarks

A `subtype` definition associates a name (`U`) with a subset of an existing type. The tags (`A` and `C` in the example) are assumed to have been defined as part of a datatype definition, with bodies which are supersets of those given in the subtype definition.

As with `nametype`, the same effect can be expressed by defining the type in terms of set comprehensions, but this can be less clear and less efficient.

A.3.5 Channels

Syntax

`channel flip, flop`	simple channels
`channel c, d : {0..3}.LEVEL`	channels with more complex protocol
`Events`	the type of all defined events

Remarks

Channels are tags which form the basis for events. A channel becomes an event when enough values have been supplied to complete it (for example `flop` above is an event). In the same way that, given

```
datatype T = A.{0..3} | ...
```

we know that `A.1` is a value of type `T`, given

```
channel c : {0..3}
```

we know that `c.1` is a value of type Event. Indeed, the channel definitions in a script can be regarded as a distributed definition for the built-in `Events` datatype.

Channels must also be rectangular in the same sense as used for datatypes. It is common in **FDR2** to make channels finite although it is possible to declare infinite channels and use only a finite proportion of them.

Channels interact naturally with datatypes to give the functionality provided by variant channels in **occam2** (and channels of variants in **occam3**). For example, given `ComplexColour` as above, we can write a process which strips out the redundant colour encodings (undoing the work performed by `make_colour`)

```
channel colour : ComplexColour
channel standard : Gun.Gun.Gun

Standardise =
    colour.RGB?x -> standard!x -> Standardise
  []
    colour.Grey?x -> standard!x.x.x -> Standardise
  []
    colour.Black -> standard!0.0.0 -> Standardise
  []
    colour.White -> standard!15.15.15 -> Standardise
```

A.3.6 Closure operations

Syntax

`extensions(`x`)`	the set of values which will 'complete' x
`productions(`x`)`	the set of values which begin with x
`{|`x_1`,`x_2`|}`	the productions of x_1 and x_2

Equivalences

```
productions(x) ≡ { x.z | z<-extensions(x)}
    {| x | ···|} ≡ Union( { productions(x) | ···} )
```

Remarks

The main use for the '`{| |}`' syntax is in writing communication sets as part of the various parallel operators. For example, given

```
channel c : {0..9}
P = c!7->SKIP [| {| c |} |] c?x->Q(x)
```

we cannot use `{c}` as the synchronisation set; it denotes the singleton set containing the channel `c`, not the set of events associated with that channel.

All of the closure operations can be used on datatype values as well as channels.

The closure operations are defined even when the supplied values are complete. (In that case `extensions` will supply the singleton set consisting of the identity value for the '`.`' operation.)

A.4 Processes

Syntax

`STOP`	no actions
`SKIP`	successful termination
`CHAOS(`a`)`	the chaotic process (on events in a)
c `->` p	simple prefix
c `?`x `?`x'`:`a `!`y `->` p	complex prefix
p `;` q	sequential composition
p `/\` q	interrupt
p `\` a	hiding
p `[[` c `<-` c' `]]`	renaming
p `[]` q	external choice
p `|~|` q	internal choice
p `[>` q	untimed timeout
p `[|` `e` `|>` q	exception
b `&` p	boolean guard
p `|||` q	interleaving
p `[|` a `|]` q	sharing
p `[` a `||` a' `]` q	alphabetised parallel
p `[` c `<->` c' `]` q	linked parallel
`;` x`:`s `@` p	replicated sequential composition
`[]` x`:`a `@` p	replicated external choice
`|~|` x`:`a `@` p	replicated internal choice (a must be non-empty)
`|||` x`:`a `@` p	replicated interleave
`[|` a' `|]` x`:`a `@` p	replicated replicated sharing
`||` x`:`a `@ [`a'`]` p	replicated alphabetised parallel
`[` c `<->` c' `]` x`:`s `@` p	replicated linked parallel (s must be non-empty)

Equivalences

As a consequence of the laws of CSP,

$$
\begin{aligned}
p \texttt{ ||| } q &\equiv p \texttt{ [| \{\} |] } q \\
\texttt{; } x\texttt{:<> @ } p &\equiv \texttt{SKIP} \\
\texttt{[] } x\texttt{:\{\} @ } p &\equiv \texttt{STOP} \\
\texttt{||| } x\texttt{:\{\} @ } p &\equiv \texttt{SKIP} \\
\texttt{[| } a \texttt{ |] } x\texttt{:\{\} @ } p &\equiv \texttt{SKIP} \\
\texttt{|| } x\texttt{:\{\} @ [}a\texttt{] } p &\equiv \texttt{SKIP}
\end{aligned}
$$

By definition

$$\texttt{CHAOS(}a\texttt{)} \equiv \texttt{|~| } x \texttt{ : } a \texttt{ @ } x \texttt{ -> CHAOS(}a\texttt{) |~| STOP}$$

Remarks

The general form of the prefix operator is cf`->`p where c is a communication channel, f a number of communication fields and p is the process which is the scope of the prefix. A communication field can be

`!`x	output
`?`x`:`a	constrained input
`?`x	unconstrained input

Note that the set a in the constrained input must match the protocol, so it must be a subset of the values that would be permitted by the unconstrained input.

Fields are processed left to right with the binding produced by any input fields available to any subsequent fields. For example, we can write

```
channel ints : Int.Int
P = ints?x?y:{x-1..x+1} -> SKIP
```

Output fields behave as suggested by the equivalence

c `!`x f `->` p $\equiv$ c`.`x f `->` p

The proportion of the channel matched by an input fields is based only on the input pattern. There is no lookahead, so if

```
channel c : {0..9}.{0..9}.Bool
P = c?x!true -> SKIP -- this will not work
Q = c?x.y!true -> SKIP -- but this will
```

then `P` is not correctly defined. The input pattern `x` will match the next complete value from the channel (`{0..9}`) and `true` will then fail to match the next copy of `{0..9}`. In the case of `@@` patterns, the decomposition is based on the left-hand side of the pattern.

If an input occurs as the final communication field it will match any remaining values, as in

```
channel c : Bool.{0..9}.{0..9}
P = c!true?x -> SKIP -- this will work
Q = c!true?x.y -> SKIP -- this will also work
```

This special case allows for the construction of generic buffers.

```
BUFF(in,out) = in?x -> out!x -> BUFF(in, out)
```

is a one place buffer for any pair of channels.

Dots do not directly form part of a prefix: any which do occur are either part of the channel c, or the communication fields. (**FDR1** took the approach that dots simply repeated the direction of the preceding communication field. This is a simplification which holds only in the absence of datatype tags.)

The guard construct 'b `&` P' is a convenient shorthand for

```
if b then P else STOP
```

and is commonly used with the external choice operator ('`[]`'), as

```
COUNT(lo,n,hi) =
  lo < n & down -> COUNT(lo,n-1,hi)
[]
  n < hi & up -> COUNT(lo,n+1, hi)
```

This exploits the CSP law that p`[]STOP`$= p$.

The linked parallel and renaming operations both use the comprehension syntax for expressing complex linkages and renamings[1]. For example,

```
p [ right.i<->left.((i+1)%n), send<->recv | i<-{0..n-1} ] q
p [[ left.i<-left.((i+1)%n), left.0<-send | i<-{0..n-1} ]]
```

Both the links (`c<->c'`) and the renaming pairs (`c<-c'`, read 'becomes') take channels of the same type on each side and extend these pointwise as required. For example,

[1] The renaming operators replace both the functional and inverse-functional renaming operations of [Hoare85]. Identifiers not targeted in a renaming are left alone.

```
p [[ c <- d ]]
```

is a process which behaves like p except all occurrences of channel c in p are replaced by channel d (so that c 'becomes' d). This is defined when `extensions(c)` is the same as `extensions(d)` and is then the same as

```
p [[ c.x <- d.x | x<-extensions(c) ]]
```

The replicated operators allow multiple generators between the operator and the '`@`' sign in the same way as comprehensions. The terms are evaluated left-to-right, with the rightmost term varying most quickly. So

```
; x:<1..3>, y:<1..3>, x!=y @ c!x.y->SKIP
```

is the same as

```
c.1.2->c.1.3->c.2.1->c.2.3->c.3.1->c.3.2->SKIP
```

The linked parallel operator generalises the chaining operator (`>>`) of [Hoare85]. For example, if `COPY` implements a single place buffer,

```
COPY(in,out) =
  in?x -> out!x -> COPY(in,out)
```

then we can implement an n-place buffer by

```
BUFF(n,in,out) =
  [out<->in] i:<1..n> @ COPY(in, out)
```

A.5 Operator Precedence

The operators at the top of the table bind more tightly than those lower down.

Class	Operators	Description	Associativity
Application	`f(0)`	function application	
	`[[ <- ]]`	renaming	
Arithmetic	`-`	unary minus	
	`*  /  %`	multiplication	left
	`+  -`	addition	left
Sequence	`^`	catenation	
	`#`	length	
Comparisons	`<  >  <=  >=`	ordering	none
	`==  !=`	equality	none
Boolean	`not`	negation	
	`and`	conjunction	
	`or`	disjunction	
Sequential	`->`	prefix	
	`&`	guard	
	`;`	sequence	
Choice	`[>`	untimed timeout	
	`/\`	interrupt	
	`[]`	external choice	
	`|~|`	internal choice	
	`[| |>`	exception	
Parallel	`[| |]  [ || ]  [ <-> ]`	parallel	none
	`|||`	interleave	
Other	`if then else`	conditional	
	`let within`	local definitions	
	`\ @`	lambda term	

The replicated versions of the process operators have the lowest predence of all. The `@@` pattern operator has a precedence just below that of function application.

Note that this table represents a simplification of the acual parser rules. For example, the parser will intrepret '`#s+1`' as '`(#s)+1`', which does not strictly agree with the table, since the parser is making (limited) use of type information.